

CampusDesk V1

Student Management System — Technical & Feature Report

React + Supabase (PostgreSQL, Auth, Storage, Row Level Security). This document describes what the application does, how each feature is implemented, what was fixed and added in the V1 stabilisation work, how it was tested, and what remains.

Project	CampusDesk — Student Management System (V1 only)
Stack	React (Create React App), @supabase/supabase-js, Supabase Postgres + Auth + Storage
Roles	Admin, Teacher, Student, Parent
Report date	9 October 2026
Deliverables	CampusDesk-V1-fixed-2.zip (full project), storage_setup.sql

Scope note. This is Version 1 only. No Kubernetes, analytics dashboards, payment gateways, notification systems, AI or other V2 features were added, and the existing architecture was kept.

Contents

- 1. Application overview and architecture
- 2. Roles and what each can do
- 3. Feature descriptions (all modules)
- 4. Fee payment system — in depth
- 5. Reports — in depth
- 6. Phone number support and teacher details
- 7. Documents and profile photos (storage)
- 8. Database, RLS and security
- 9. Seed / test data
- 10. Code structure and key files
- 11. Testing performed and results
- 12. Setup guide
- 13. Changes made, file by file
- 14. Known limitations

1. Application overview and architecture

CampusDesk is a single-page React application that talks directly to Supabase. There is no custom backend server. Supabase provides authentication (email + password), the Postgres database, file storage, and — critically — Row Level Security (RLS), which enforces who may read or write each row.

- **Frontend:** React with route-based pages under `src/pages`, grouped as admin, shared (used by several roles), and account.
- **Data layer:** small query modules in `src/lib/queries` (students, teachers, classes, fees, attendance, exams, leave, notices, events, timetable, academic report, analytics, audit, activity, me, account). Pages never build SQL; they call these functions.
- **Auth:** `AuthContext` loads the session and the user's profiles row, exposing `user`, `role`, `profile`. Routes are guarded by role.
- **Security:** the browser only holds the public anon key. Every table has RLS policies; the database, not the UI, decides access.
- **Admin user creation:** a second, non-persisting Supabase client (`adminActionClient.js`) calls `signUp` so the admin is not logged out when creating a student, teacher or parent account.

Main tables

Table	Purpose
profiles	One row per auth user: <code>full_name</code> , <code>email</code> , <code>phone</code> , <code>role</code> , <code>status</code> , <code>avatar_url</code>
students / teachers	Role details. students: <code>roll_no</code> , <code>dob</code> , <code>gender</code> , <code>address</code> , <code>admission_date</code> , <code>class_id</code> , <code>parent_id</code> . teachers: <code>employee_id</code> , <code>department</code> , <code>qualification</code> , <code>joining_date</code>
classes / subjects / teacher_subjects	Classes (<code>name</code> , <code>section</code> , <code>academic_year</code> , <code>class_teacher_id</code>), their subjects, and which teacher teaches which subject
attendance / marks / exams / results	Per-subject attendance, marks per exam, exam definitions, computed and published results
fee_structure / fee_payments	Fee items per class; payment rows with status <code>paid</code> / <code>partial</code> / <code>due</code> / <code>pending_verification</code> / <code>rejected</code>
leave_requests, notices, events, timetable, teacher_queries, audit_logs	Remaining V1 modules

2. Roles and what each can do

Role	Capabilities
Admin	Full management: students, teachers, classes, subjects, teacher assignment, timetable, fee structure, manual payment entry, verification of submitted payments, exams and publishing results, notices, events, leave overview, audit log, full student and teacher reports, institution overview.
Teacher	Sees assigned classes/subjects; marks attendance; enters marks for own subjects; sees timetable; approves or rejects student leave; replies to student/parent queries; views the shared academic report for their classes. Cannot see fees and cannot publish results.
Student	Own dashboard, attendance history, published results, fee status and payment submission, academic report, leave requests, notices/events, timetable, documents, teacher queries, profile and photo.
Parent	The same views for their linked child (via students.parent_id), including fee status and payment submission; own leave requests and documents.

3. Feature descriptions

Authentication and account

Email/password login, password reset by email token, change password, and a Settings page to edit own name and phone and upload a profile photo. A trigger creates the profiles row on sign-up; role changes are admin-only (a database trigger blocks self-escalation).

Student management

List, search, add, edit and suspend/activate students. Add form captures name, email, password, phone, roll number, class, DOB, gender, address, and can create and link a parent in the same flow. The detail page shows all fields, the linked parent (with phone), and lets the admin link an existing parent by email or create a new one.

Teacher management

List, add, edit, suspend/activate teachers with name, email, phone, employee ID, department, qualification and joining date. The detail page and full report show identity details and teaching assignments.

Classes and subjects

Admin creates classes with section and academic year and picks a class teacher (classes.class_teacher_id). Subjects belong to a class; teachers are assigned to subjects (teacher_subjects).

Attendance

Teachers pick class, subject and date and mark present / absent / late for the roster; saving is an upsert on (student, subject, date). Students and parents see history; admin can browse by class and date.

Exams, marks and results

Admin creates exams per class and term. Teachers enter marks for their own subjects only. Admin computes and publishes results (total, percentage, grade, rank). Students and parents see only published results.

Fees

Admin defines fee items per class with amount and due date, can record payments manually, and verifies payments submitted by students/parents. See section 4.

Notices and events

Admin posts notices (targeted to a role or class, pinnable, with expiry) and events; everyone sees what applies to them. The dashboard shows recent activity.

Leave

Students, parents and teachers submit leave requests; teachers and admins approve or reject; requesters see decisions.

Timetable

Admin builds a weekly timetable per class (subject, teacher, day, time); teachers see their own schedule; students see their class.

Documents

Each user uploads ID proofs, certificates and similar files by category, views them via short-lived signed links and deletes them. Admin can see any user's documents. Required categories per role are shown as a checklist.

Chat / queries

Students and parents send typed queries to a teacher of their class; teachers reply. This was left unchanged in V1.

Audit log

Best-effort log of admin actions viewable by admin.

Analytics widgets

Existing student attendance/results/fee summaries and institution overview. The fee numbers now use the shared fee calculation so they match the Fees page.

4. Fee payment system — in depth

4.1 The problem that was fixed

Fee numbers were calculated separately in several places, a student could submit another payment while one was still awaiting verification, an admin approval could push the total past the fee amount, and the display could show values such as “₹45300 of ₹45000 paid”. The admin page showed too little to verify a request confidently.

4.2 One definition of a fee balance

A new module, `src/lib/feeCalc.js`, defines every fee number once. All screens (student/parent Fees, admin Fees and reports, academic report, dashboards, and the submit/approve validation) call it.

Payment status	Meaning	Counts toward paid?
paid / partial	Money confirmed by an admin	Yes
pending_verification	Submitted by student/parent, waiting for an admin	No — shown separately
rejected	Declined by an admin	No
due	Legacy placeholder	No

- **confirmedPaid** = sum of paid/partial rows, capped at the fee amount so old over-recorded data can never display above the fee.
- **remaining** = $\max(0, \text{fee amount} - \text{confirmedPaid})$. Pending money is never subtracted.
- **status** = paid when remaining is 0; partial when some is confirmed; otherwise due. Pending never changes it.
- **canPay** = remaining > 0 **and** no pending request exists.

4.3 The four displayed states

State	Example	Display	Pay button
Fully paid	Fee ₹45,000, confirmed ₹45,000	₹45000 of ₹45000 paid — status paid	Hidden
Partially paid	Fee ₹45,000, confirmed ₹20,000	₹20000 of ₹45000 paid · ₹25000 remaining — partial	Shown
Unpaid	Fee ₹2,00,000, confirmed ₹0	₹0 of ₹200000 paid · ₹200000 remaining — due	Shown
Pending verification	Unpaid + ₹2,000 submitted	₹0 of ₹200000 paid · ₹200000 remaining · 1 payment awaiting verification, plus amount, mode, date, reference	Hidden

4.4 Submitting a payment (student or parent)

`submitPaymentRequest` performs these checks in order and inserts the pending row only if all pass:

- Amount is numeric and greater than 0.
- The fee item exists (its amount is read from the database, never trusted from the page).
- The fee belongs to the student's class.
- The fee is not already fully paid.
- No pending request already exists — otherwise: “A payment is already pending verification for this fee. Please wait for the administrator to verify it.”
- Amount does not exceed the remaining balance — otherwise: “You can pay a maximum of ₹25000.” (with the real remaining figure).

A second safety net lives in the database: a partial unique index allows only one pending row per student per fee, so two browser tabs (or a student and parent at the same moment) cannot both succeed. The resulting database error is translated into the same friendly message.

4.5 Admin verification

The admin Fees page lists every pending request with student name, roll number, class, fee type, fee amount, submitted amount, mode, date, reference and status, with Approve and Reject buttons.

- The row is re-read from the database before acting; if it is no longer pending the admin sees “This payment has already been processed.”
- Approve re-validates against the confirmed total of the other payments, refuses if it would exceed the fee, and then **updates the existing row** to paid or partial. No new row is created.
- The update only applies to a row that is still pending, so a double click or two admin tabs cannot approve twice.
- Reject sets status to rejected; it never counts and Pay returns if a balance remains and nothing else is pending.
- Buttons are disabled while one decision is in flight.

4.6 Admin manual payment entry

recordPayment uses the same validation (mode “record”): amount must be valid and cannot exceed the remaining balance; status is derived (paid or partial). The amount input is capped at the remaining balance and the preview shows confirmed paid, remaining and pending (not counted).

4.7 Overpayment protection — three layers

- **UI:** input max equals remaining; remaining is never negative.
- **Query layer:** every creation/confirmation path validates against the current database state.
- **Database:** a trigger (fee_payment_guard_patch.sql) rejects any row that becomes paid/partial if it would take the confirmed total above the fee. It fires only on inserts and on changes of status or amount, so historical rows are preserved untouched.

A commented read-only query in the patch lists existing over-paid records so they can be reviewed manually; nothing was deleted.

5. Reports — in depth

5.1 Admin: Student Full Report (/students/:id/report)

Rebuilt into clearly headed sections, all using the existing panel / details-card / recent-row styles:

Section	Content
STUDENT INFORMATION	Name, Student ID (the profile UUID — the app's only ID), roll number, email, phone, class, section, academic year, date of birth, gender, address, admission date, account status
PARENT / GUARDIAN	Name, parent ID, email, phone, account status, relationship (linked through students.parent_id). Shows “No parent linked” when none; a distinct message if a parent exists but is not readable
CLASS TEACHER	Name, employee ID, email, phone, department — via classes.class_teacher_id. Shows “No class teacher assigned” or “No class assigned”
ATTENDANCE	Per subject: present, absent, late, total sessions, percentage (existing breakdown kept and extended with the total)
MARKS	Subject, exam, term, teacher, marks obtained, maximum, percentage
FEES	Per fee: total, confirmed paid, remaining, due date, status, pending count and amount, with pending date/mode/reference shown separately and labelled “awaiting verification”
LEAVE REQUESTS / DOCUMENTS	Existing sections kept (dates, reason, status; uploaded documents)

5.2 Student and parent: Academic Report (/report)

The shared AcademicReport page keeps its architecture (student, parent and teacher views). The student and parent view now starts with a **Student profile summary** (name, ID, roll, class, section, academic year, email, phone), the **class teacher** (name and phone), and **Parent information** (name, email, phone), then keeps Attendance and Marks, and ends with a compact **Fees summary**: total, paid, remaining, status and pending verification amount. A parent sees only the linked child.

Each section loads independently: if one query is refused or fails, an error line appears and the other sections still render, instead of a blank page. Missing values show “—”, never undefined, null or NaN.

5.3 Shared loader

getStudentReportProfile in students.js fetches the student, profile, parent and class teacher in small separate queries. This keeps the page working when RLS hides a related row, because the missing piece becomes an empty section rather than collapsing the whole result.

6. Phone support and teacher details

- profiles.phone was already in the schema. It is now read everywhere it is relevant and editable for students, parents and teachers.
- Add Student / Add Teacher forms and the Student and Teacher detail edit forms include a phone field; creating a parent (inside Add Student or from the student detail page) also accepts a phone.
- Teacher detail and Teacher Full Report show name, employee ID, email, phone, department, qualification, joining date and status.
- Own phone can be edited in Settings.

7. Documents and profile photos (storage)

The UI had upload buttons but no storage buckets existed, and the original policy file assumed they had been created manually. `supabase/storage_setup.sql` now creates them and their policies in one idempotent script.

Bucket	Visibility	Limit / types	Access
avatars	Public	2 MB; jpeg, png, webp, gif	Anyone can read; users write only their own folder
documents	Private	10 MB; pdf, jpeg, png, webp, doc, docx, txt	Owner and admin only; users may upload, view and delete their own
receipts	Private	5 MB; pdf, images	Owner reads, admin manages (reserved; not used by V1 screens)

Files are stored as `<user-id>/<file>`; the first folder is checked against the signed-in user.

Bugs fixed in the storage code

- **Delete silently did nothing:** there was no delete policy for documents, and storage reports success with an empty result. The policy was added and the app now raises an error when nothing was removed.
- **File names rejected by storage:** characters such as `[ ] # %` or accents caused upload failures. Names are now sanitised before upload.
- **Validation and messages:** size and type are checked before upload; a missing bucket tells the user to run `storage_setup.sql`; permission errors are explained; upload and delete show a success message; the same photo can be selected twice; stale avatars with a different extension are cleaned up.

8. Database, RLS and security

8.1 SQL files and order

#	File	What it does
1	<code>supabase_schema.sql</code>	Tables, auto-profile trigger, base RLS policies
2	<code>feature_additions_patch.sql</code>	Fee verification states and related additions
3	<code>leave_requests_patch.sql</code>	Lets teachers see student leave requests
4	<code>profiles_rls_patch.sql</code>	Rewritten — scoped profile reads and role/status protection (see below)
5	<code>fee_payment_guard_patch.sql</code>	New — one pending per fee, no new overpayment, positive amounts
6	<code>storage_setup.sql</code>	New — buckets and storage policies (replaces <code>storage_policies.sql</code>)
7	<code>seed_dummy_data.sql</code>	Optional test data; run as one script

8.2 Profiles RLS change

The earlier patch let every signed-in user read every profile. Because profiles now holds phone numbers, that would let Student A read Student B's email and phone. The new patch replaces it with scoped policies:

- Everyone signed in can read **teacher** profiles (school directory).
- A parent can read their **own linked child** only.
- A student can read their **own parent** only.
- Teachers keep read access to student profiles (rosters, attendance, marks, leave) — teacher permissions are unchanged.
- Own profile and admin full access remain from the base schema.
- A trigger stops non-admins changing their own **role or status** through the profile-update policy.

8.3 Fee payment table protections

- Unique partial index: one pending_verification row per (student, fee).
- Check: amount_paid > 0 (not validated against old rows).
- Trigger: confirmed total may not exceed the fee amount for new or changed confirmed rows.

9. Seed / test data

The seed script creates only dummy @campusdesk.test-style users and does not touch real accounts. It was extended to cover every case required for testing:

Scenario	Where in the seed
Fully paid tuition	Students whose sequence number is a multiple of 3
Partially paid tuition	Students with sequence 1 mod 3
Pending verification	Students with sequence 2 mod 3
Partial plus pending (no Pay)	Student 7: ₹5,000 confirmed + ₹3,000 pending
Rejected payment only	Student 8: still due, Pay visible
Transport fee: paid vs completely unpaid	Student 9 paid in full; others unpaid
Student without parent	Students 23 and 24
Class without class teacher	Class 11-B
Phones	All teachers; most students (not every 6th); all parents except one — to test the “—” display

Sample logins from the seed follow the pattern first-name email with password First-name plus 1234 (for example aarav@gmail.com / Aarav1234; parent aaravparent@gmail.com; teacher anita@gmail.com / Anita1234).

10. Code structure and key files

File	Role
src/lib/feeCalc.js	Single source of truth for fee math, validation and display helpers
src/lib/queries/fees.js	Fee structure CRUD, balance, record payment, submit request, pending list, verify (approve/reject)
src/lib/queries/students.js, teachers.js	CRUD with phone support; getStudentReportProfile loader
src/lib/queries/analytics.js	Dashboard totals now confirmed-only via feeCalc
src/lib/storage.js	Avatar/document upload, list, signed URL, delete with validation
src/components/ReportSections.js	Shared Student/Parent/Class-teacher cards and fee sections
src/pages/shared/Fees.js	Student/parent fee screen with pending UI and submit form
src/pages/admin/AdminFees.js	Fee structure, manual payment, pending verification queue
src/pages/admin/StudentFullReport.js	Expanded admin report
src/pages/shared/AcademicReport.js	Student/parent/teacher academic report
src/pages/admin/AddStudent, AddTeacher, StudentDetail, TeacherDetail, TeacherFullReport	Phone fields and identity details
src/pages/shared/Documents.js, pages/account/Settings.js	Upload UIs with validation and messages
supabase/*.sql	Schema, patches, storage setup, seed

11. Testing performed and results

All results below were actually run. Tests used a local Postgres loaded with the project's SQL files, plus the real PostgREST API, standing in for Supabase.

Test area	What was checked	Result
Unit — fee calculation	Matrix tests 1-10: paid, partial, due, pending, partial+pending, rejected, legacy overpaid data, bad numbers; validation messages incl. ₹25,001 blocked	Pass
Flow — duplicate payment	Submit → pending → refresh → duplicate rejected → parent sees same → admin rejects → Pay returns → new payment → approve → confirmed total updates	Pass
Flow — verification safety	Same payment cannot be approved twice; approval that would overpay refused; approval updates existing row	Pass
UI — Fees page (student and parent)	Pending message and details shown, Pay hidden for pending/paid, visible only for partial/unpaid, error shown on failure, no undefined/null/NaN	Pass
UI — report sections	All fields present; “—”, “No parent linked”, “No class teacher assigned” states	Pass
Storage helpers	Name sanitising, size/type checks, missing-bucket and permission messages, empty-delete detected	Pass
Database — RLS isolation	Student A / Parent A cannot read Student B / Parent B profiles, fees, attendance or marks; cannot submit for others; cannot insert confirmed payments or self-approve; cannot change own role; teacher access unchanged	Pass
Database — fee guards	Duplicate pending, confirmed overpay, approval overpay and zero amount all rejected	Pass
Database — storage policies	Cross-user upload/read/delete blocked; own delete works; admin access works; script re-runs without error	Pass
Integration — real queries through RLS	46 checks as admin, student, parent and teacher: every list/report screen, add/edit/delete flows, fee workflow, attendance and marks, leave and query reply; teacher blocked from publishing results and fees	46 / 46 pass

- **npm test -- --watchAll=false:** 116 tests pass across 6 suites (70 unit/UI + 46 integration; the integration suite runs only when a test API URL is provided).
- **npm run build:** “Compiled successfully”, no warnings.
- The obsolete App.test.js (it imported a component that no longer exists) was removed and replaced.

12. Setup guide

- Create a Supabase project; in Project Settings → API copy the Project URL and the anon/publishable key (never the service_role key).
- Create .env in the project root: REACT_APP_SUPABASE_URL and REACT_APP_SUPABASE_ANON_KEY; restart the dev server after any change.
- Run the SQL files in the order given in section 8.1.
- Create the admin: Authentication → Users → Add user (auto-confirm), then update profiles set role = 'admin' where email = '...';
- npm install, then npm start (http://localhost:3000).
- Optionally run seed_dummy_data.sql for sample students, parents, teachers and fees.

13. Changes made, file by file

File	Change
feeCalc.js (new)	Shared fee calculation, validation and display helpers
queries/fees.js	Rewritten on feeCalc; DB-sourced fee amount; class check; duplicate and overpay checks; safe approve/reject

File	Change
shared/Fees.js	Pending details and message, Pay hidden when pending/paid, reload after submit, error and success states
admin/AdminFees.js	Rich pending-request rows, safe verification, balance preview with pending, capped input
queries/analytics.js	Confirmed-only fee totals
queries/students.js / teachers.js	Phone read and write; report profile loader; parent creation with phone
ReportSections.js (new)	Reusable report cards and fee sections
admin/StudentFullReport.js	Student, parent, class teacher and detailed fee sections
shared/AcademicReport.js	Profile summary, class teacher, parent, compact fees, per-section error handling
Add/Detail pages for students and teachers	Phone fields and extra identity details
storage.js, Documents.js, Settings.js	Validation, sanitised names, safe delete, messages
profiles_rls_patch.sql	Scoped profile reads and role/status protection
fee_payment_guard_patch.sql (new)	Fee guards in the database
storage_setup.sql (new)	Buckets and policies; storage_policies.sql removed
seed_dummy_data.sql	Extra fee, phone, no-parent and no-class-teacher cases
Tests (new)	feeCalc, fees flow, Fees page, ReportSections, storage and integration suites
README.md	Updated SQL run order

14. Known limitations

- The test database used a stand-in for Supabase auth and storage, so the SQL should be run once in the real Supabase editor and the first real upload tried manually.
- Account creation (Add Student/Teacher/Parent) relies on Supabase sign-up, which could not be run in the test environment — try Add Student once with the admin login.
- Existing over-paid rows are preserved and only capped in the display; review them with the commented query in the guard patch.
- The unique-pending index will fail to create if live data already has two pending rows for one fee — resolve them in Admin Fees first.
- Bulk student import does not read a phone column.
- The receipts bucket is created but unused by V1 screens.
- The app was not clicked through in a browser; UI behaviour was verified with automated component tests.